



华中科技大学

函数式编程原理课程报告

姓 名：潘宇璇
班 级：智能 2302 班
学 号：U202315695
指导教师：顾琳

分数	
教师签名	

2025 年 11 月 6 日

目 录

一、 Heapify 求解	1
1.1 问题需求	1
1.2 解题思路与代码	3
1.3 遇到的问题及运行结果	6
1.4 性能分析（请用树的深度进行分析）	8
二、 函数式拓展学习调研	9
2.1 函数式程序应用场景	9
2.2 函数式特征延伸	10
2.3 高阶与多态函数	10
三、 函数式编程学习建议与心得	9

一、Heapify 求解

1.1 问题需求

实验题目

一棵 minheap 树定义为:

- t is Empty;
- t is a Node(L, x, R), where R, L are minheaps and $\text{value}(L), \text{value}(R) \geq x$ ($\text{value}(T)$ 函数用于获取树 T 的根节点的值)

编写函数 treecompare, SwapDown 和 heapify:

treecompare: tree * tree -> order

(* when given two trees, returns a value of type order, based on which tree has a larger value at the root node *)

SwapDown: tree -> tree

(* REQUIRES the subtrees of t are both minheaps)

(ENSURES swapDown(t) = if t is Empty or all of t's immediate children are empty then * just return t, otherwise returns a minheap which contains exactly the elements in t. *)

heapify : tree -> tree

(* given an arbitrary tree t, evaluates to a minheap with exactly the elements of t. *)

需求分析

本次实验要求在 Standard ML 语言环境下实现一个纯函数式的 min-heapify 框架。但是需要先说明的是，题目预设了一套完整的前置辅助函数与数据定义，其中也有部分函数如 trav、listToTree、binarySearch 等是其引导下，我对照题目要求自己编写的。为体现整个设计流程，我将首先对核心任务之前的所有基础定义进行梳理，再逐步展开三项核心子任务。

基础设施与数据定义

tree 数据类型:

<code>datatype tree = Empty Br of tree * int * tree</code>
--

采用二叉树代数类型，节点携带整型键值，左右子树可为空。该定义贯穿后续全部操作，故在此特予说明。

输入/输出原语：

`getInt ()`：从标准输入扫描一个 `int`；

`getIntList n`：连续读取 `n` 个整数并返回列表，用于快速构造测试数据；

`printInt / printIntList`：行缓冲输出，方便立即观察结果。

辅助函数大致有这一些：

`split`：将列表按奇偶索引拆成两个近似等长子列表，使得接下来 `listToTree` 生成的二叉树尽可能平衡；

`listToTree (int list -> tree)`：利用 `split` 函数的结果，递归地把列表转为完全二叉树（非堆），会作为后续 `heapify` 的输入等等；

`trav`：中序遍历，用于显示表明节点在堆化前后的顺序；

`binarySearch (tree * int -> bool)`：。当输入参数 1 为有序树时，如果树中包含值为输入参数 2 的节点，则返回 `true`；否则返回 `false`。

核心任务

核心任务可分解为以下三个子函数：

1. `treecompare : tree * tree -> order`

比较两树根键值，空树视为最小。

2. `SwapDown : tree -> tree`

前置：左右子树均为最小堆；

功能：使最后输出的整棵树都满足父节点值 $\leq$ 子节点值，也就是返回一个恰好包含 `t` 中元素的最小堆。

3. `heapify : tree -> tree`

给定任意树 `t`，返回一个恰好包含 `t` 中元素的最小堆。

辅助验证

为方便调试、观察堆形状变化以及递归验证一棵树是否为最小堆，我决定进行以下函数的定义：

`isMinHeap : tree -> bool` —— 递归验证父子键值偏序，输出是否为最小堆；

`showTree : tree -> string` —— 括号式先序遍历输出树形状，以直观观察堆形状变化。

1.2 解题思路与代码

总体思路

首先在测试阶段，我对一些刻意设计的列表 `listToTree` 先得到完全二叉树，再用 `heapify` 递归，而其内部调用 `SwapDown` 完成根节点下沉，最后交由 `isMinHeap + showTree` 验证。

验证成功后我改用 `getIntList` 获取的输入列表取代测试阶段的列表，`printIntList` 输出 `trav` 遍历后的结果。

粗略叙述这几个函数的思路如下：

`listToTree` 和 `split`:

用于将列表构造为完全二叉树（非 BST，非 heap），是构建测试数据的方式，不影响堆逻辑本身，但它不保证输入是堆，这是后续的 `heapify` 要解决的问题。

`trav`（中序遍历）：

用于打印结果，不影响堆逻辑。

`treecompare : tree * tree -> order`

对两棵二叉树进行根节点键值比较，返回 SML 内建类型 `order`（`LESS | EQUAL | GREATER`）。当两棵树均为 `Empty` 时视为相等；任何非空树均大于空树。

`SwapDown : tree -> tree`

前置条件：输入树的左、右子树已经满足 `min-heap` 性质。

功能：将根节点键值与其直接子根中的较小者不断交换，直至整棵树符合父 $\leq$ 子的约束。若输入为空或叶子节点，则直接返回自身。

`heapify : tree -> tree`

对任意形状的二叉树做完全堆化，先自底向上递归调用 `heapify` 以保子树合法，再借助 `SwapDown` 将当前根节点沉到合适深度，最终输出一棵包含原树全部节点且高度不增的最小堆。`SwapDown`

绘制流程图如下：

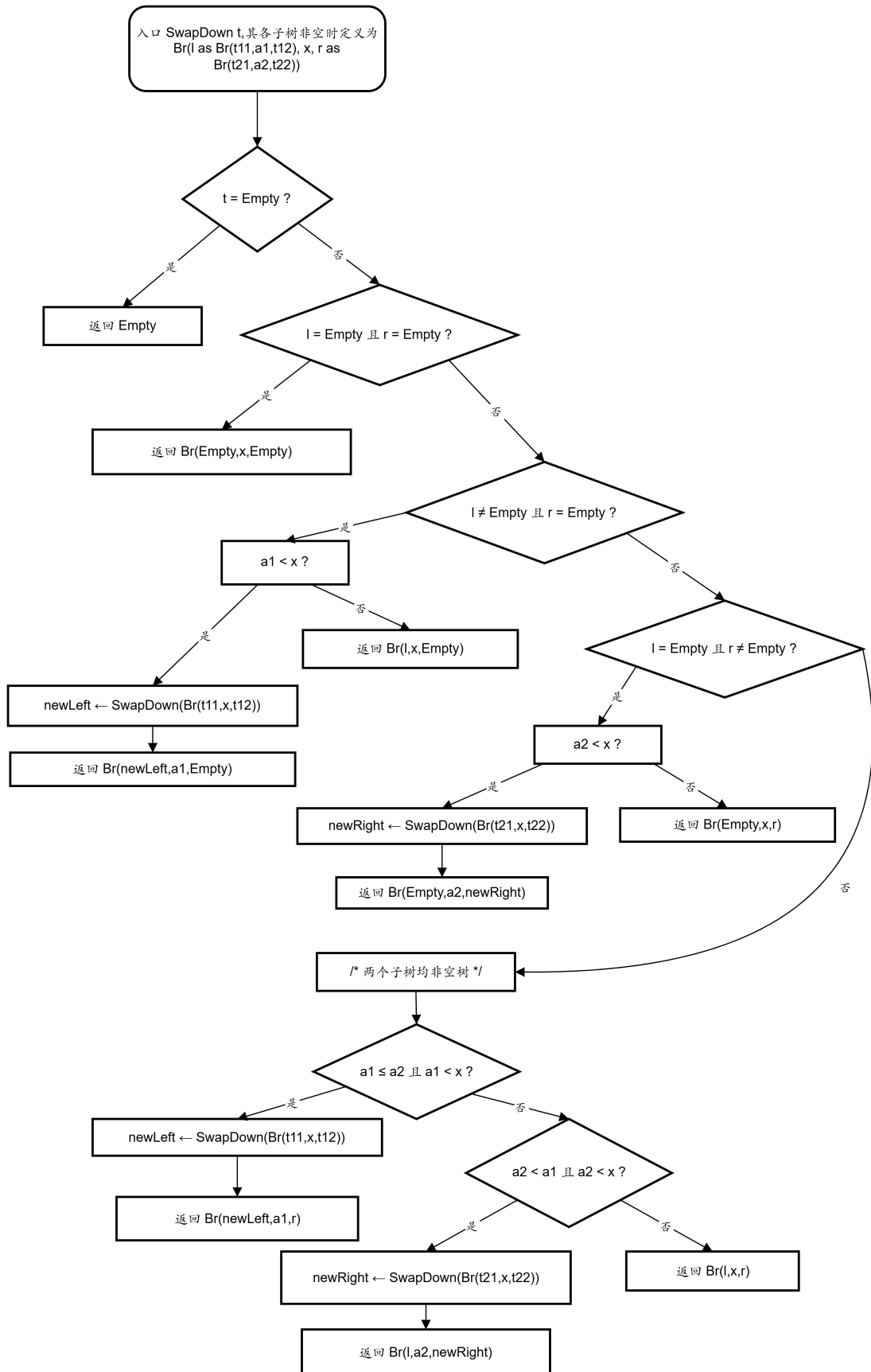


图 1 SwapDown 设计流程图

总结一下，该模式按空树、左右子树都空、仅左子树空、仅右子树空、左右子树都非空拆成 5 种情况分类讨论处理：

① $t = \text{Empty}$

无需下沉，直接返回 Empty 。

② $\text{Br}(\text{Empty}, x, \text{Empty})$

已经没孩子，无需下沉，直接返回原输入 $\text{Br}(\text{Empty}, x, \text{Empty})$ 。

③ $\text{Br}(\text{l as Br}(t11, a1, t12), x, \text{Empty})$ （只有左子树）

若 $a1 < x$ ：

先构造临时树 $\text{Br}(t11, x, t12)$ （把 x 放到左子树根位置）；

对这棵临时树 再调用 SwapDown ，保证左子树内部依旧合法；

用返回的 newLeft 拼成最终结果 $\text{Br}(\text{newLeft}, a1, \text{Empty})$ 。

否则保持不动。

④ $\text{Br}(\text{Empty}, x, r \text{ as Br}(t21, a2, t22))$

与③同理。

⑤ $\text{Br}(\text{l as Br}(t11, a1, t12), x, r \text{ as Br}(t21, a2, t22))$

先比较 $a1$ 、 $a2$ 取较小者；若该较小者也小于根节点键值 x 则交换，并对相应子树递归下沉；若 x 已最小则终止。

heapify

设定函数逻辑相对简单，有 SwapDown 的辅助下较易完成，在此列出代码，稍作阐释：

空树的情况很容易理解。最主要是非空树的情况下，由于输入的任意性，左右子树均不一定是最小堆，所以必须先对左右子树进行最小堆化，此即自底向上的递归调用。而子树堆化后，其根 x 又可能破坏堆性，故在左右子树都最小堆化后就可以统一交由 SwapDown 修复，以期达成整棵树的最小堆性质。

```
fun heapify( $\text{Empty}$ ) =  $\text{Empty}$ 
  | heapify( $\text{Br}(t1, x, t2)$ ) =
    let
      val left = heapify( $t1$ )
      val right = heapify( $t2$ )
    in
       $\text{SwapDown}(\text{Br}(\text{left}, x, \text{right}))$ 
    end;
```

1.3 遇到的问题及运行结果

仅依靠中序遍历无法定位形状错误

随堂实现之初，我写下的 `SwapDown` 算法思路比较潦草，有些逻辑错误，但是由于头歌检验机制的问题，仅用中序遍历输出结果，未编写可视化与验证函数，无法定位形状错误，导致我无法正确判断是否符合要求，也无法进行调试。

所以后续书写报告的时候，重新设计了实验，增加了以下函数，他们的功能如前文章节——辅助验证所示。`showTree` 以明确的格式“根(左子树)(右子树)”帮助我检查了树被处理后的结构，而 `isMinHeap` 的递归断言、逐层检验能够直接断定某一测试集下的答案正确与否，帮助了我进行批量测试和检错。后者的思路也与 `SwapDown` 类似的分情况讨论，较易理解，不再赘述。

代码相对简洁，展示如下。

```
fun showTree Empty = "E"
  | showTree (Br(l, x, r)) =
    Int.toString x ^ "(" ^ showTree l ^ ")" ^ showTree r ^ ")"

fun isMinHeap(Empty) = true
  | isMinHeap(Br(Empty, x, Empty)) = true
  | isMinHeap(Br(Br(t11, a1, t12), x, Empty)) =
    x <= a1 andalso isMinHeap(Br(t11, a1, t12))
  | isMinHeap(Br(Empty, x, Br(t21, a2, t22))) =
    x <= a2 andalso isMinHeap(Br(t21, a2, t22))
  | isMinHeap(Br(Br(t11, a1, t12), x, Br(t21, a2, t22))) =
    x <= a1 andalso x <= a2 andalso
    isMinHeap(Br(t11, a1, t12)) andalso isMinHeap(Br(t21, a2, t22));
```

```
175 end
176 (* 自己的调试专用: *)
177 val t0 = listToTree [99,98,91,100,5,4,3,2,1,0,9];
178 val t1 = heapify t0;
179 print ("before: " ^ showTree t0 ^ "\n");
180 print ("after: " ^ showTree t1 ^ "\n");
181 val _ = if isMinHeap t1 then print "PASS: it is a min-heap!\n"
182         else print "FAIL: not a min-heap!\n";

> val t0 = Br (Br (Br (Empty, 100, Br (Empty, ...)), 98, Br (Empty, 4, ...)), 99, Br (Br (Empty, 5, Br (Empty, ...)), 91, Br (Empty, ...)): tree;
> val t1 = Br (Br (Br (Empty, 99, Br (Empty, ...)), 2, Br (Empty, 4, Br ...)), 1, Br (Empty, ...)): tree;
> val it = (): unit;
Printed: before: 99(98(100(E)(2(E)(E))) (4(E)(0(E)(E)))) (91(5(E)(1(E)(E))) (3(E)(9(E)(E))))
> val it = (): unit;
Printed: after : 0(2(99(E)(100(E)(E))) (4(E)(98(E)(E)))) (1(5(E)(91(E)(E))) (3(E)(9(E)(E))))
Printed: PASS: it is a min-heap!
```

图 2 可视化调试过程

在图 2 中，我设计了一个用于测试的列表，可以看到最初的大数字都在顶层，而 `heapify` 后得到了一个最小堆的序列。其结果的括号匹配后标注颜色如下，每个颜色的括号对应同一层级的子树/节点：

0(2(99(E)(100(E)(E)))(4(E)(98(E)(E))))(1(5(E)(91(E)(E)))(3(E)(9(E)(E))))

采用在线工具可以绘制出来其图像如下，正确性得到验证：

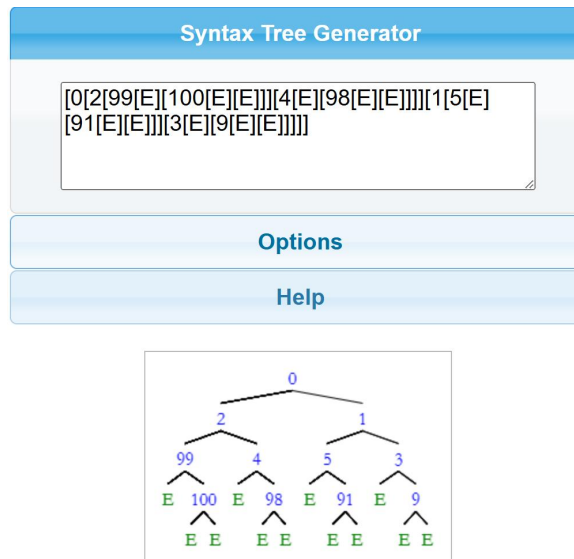


图 3 满足最小堆性质的二叉树图示

单子树情况下对新子树重新下沉步骤的缺失

该错误是我回头进行逻辑检查的时候所发现，其非常隐秘，因为课堂上 listToTree 生成的特性导致其错误无法暴露。一开始我的 SwapDown 在单子树分支下写为：

```
SwapDown(Br(Br(tl1, a1, t12), x, Empty)) =  
    if a1 < x then Br(Br(tl1, x, t12), a1, Empty)  
    else Br(Br(tl1, a1, t12), x, Empty)
```

但是后来发现其缺少递归调用，没有对新子树重新下沉（但我当时在两个子树都不为空的情况下做了递归），而我多个测试用例都没有暴露出这一缺陷，我后来仔细分析，发现由于题目的前置设定，使得 listToTree 函数所构造的二叉树都只有倒数第二层才会出现单边子树为空的现象，也就是说不会由于这一分支的错误（到单子树为空的地方 swapdown 这个分支只会调用一次，而因为构造的原因，倒数第二层也只用调用一次 swapdown）出现爷爷>孙子的情况。使得这一错误很难在实际用例中得到体现。

因此我直接构造了一个树，设计了这些分支使得右子树全程空且逆序，输入 heapify 函数后果然发现出错。

```
175 val bad = Br(Br(Br(Br(Empty,1,Empty), 2, Empty), 3,  
    Empty), 4, Empty);  
176 val t1 = heapify bad;  
177 print ("before: " ^ showTree bad ^ "\n");  
178 print ("after: " ^ showTree t1 ^ "\n");  
179 val _ = if isMinHeap t1 then print "PASS: it is a min-  
    heap!\n" else print "FAIL: not a min-heap!\n";  
  
Printed: PASS: it is a min-heap!  
> val bad = Br (Br (Br (Br (Empty, ...), 2, Empty), 3, Empty), 4, Empty): tree;  
> val t1 = Br (Br (Br (Br (Empty, ...), 3, Empty), 4, Empty), 1, Empty): tree;  
> val it = (): unit;  
Printed: before: 4(3(2(1(E)(E))(E))(E))(E)  
> val it = (): unit;  
Printed: after: 1(4(3(2(E)(E))(E))(E))(E)  
Printed: FAIL: not a min-heap!
```

图 4 诱导错误函数出错

于是我对单边分支增加继续下沉的过程，也就是交换后构造临时树并再次调用 SwapDown，使根节点能够一路下沉不停留。相应地，有以下分支：

```
SwapDown (Br(l as Br(tl1,a1,tl2), x, Empty)) =
  if a1 < x then
    let val newLeft = SwapDown(Br(tl1,x,tl2))
    in Br(newLeft, a1, Empty) end
  else
    Br(l, x, Empty)
```

如图 5，修正后同一用例通过验证，深度保持不变。

<pre>val bad = Br(Br(Br(Empty,1,Empty), 2, Empty), 3, Empty), 4, Empty); val t1 = heapify bad; print ("before: " ^ showTree bad ^ "\n"); print ("after : " ^ showTree t1 ^ "\n"); val _ = if isMinHeap t1 then print "PASS: it is a min- heap!\n" else print "FAIL: not a min-heap!\n";</pre>	<pre>Printed: PASS: it is a min-heap! > val bad = Br (Br (Br (Br (Empty, ...), 2, Empty), 3, Empty), 4, Empty): tree; > val t1 = Br (Br (Br (Br (Empty, ...), 3, Empty), 2, Empty), 1, Empty): tree; > val it = (): unit; Printed: before: 4(3(2(1(E)(E))(E))(E))(E) > val it = (): unit; Printed: after : 1(2(3(4(E)(E))(E))(E))(E) Printed: PASS: it is a min-heap!</pre>
---	---

图 5 修正后同一用例通过验证

1.4 性能分析（请用树的深度进行分析）

量化分析程序的串并行情况和时间开销。

对 n 个节点的完全树， $T(n) = 2T(n/2) + O(h) \Rightarrow$ 总体 $O(n)$ 。

串并行情况：

首先对于并行性而言，heapify 会对左右两个子树不断分支，最后调用 SwapDown 进行调整，那么“不断分支”的这一过程，就完全可以并行。

而单个 SwapDown 进程是顺序向下执行的，这一过程内部属于串行。多个 SwapDown 之间的情况更复杂一些，只有被不同 heapify 调用的才会并行。

时间开销：

设 $T_s(h)$ 为对高度 h 子树执行 SwapDown 的代价：

$$T_s(h) = T_s(h-1) + O(h) \Rightarrow T_s(h) = O(h)$$

设 $T_h(n)$ 为对 n 节点完全树执行 heapify 的代价：

$T_h(n) = 2 \cdot T_h(n/2) + T_s(h)$ 其中 $h = \lceil \log_2 n \rceil$ ，因为长度为 n 的整数列表，经 listToTree 生成近似完全二叉树，高度 $h = \lceil \log_2(n+1) \rceil$ 。

由主定理：

$$T_h(n) = 2 \cdot T_h(n/2) + O(\log n) \Rightarrow T_h(n) = \Theta(n)$$

串行分析：

执行 heapify 的递归栈深度 = h ，内存开销 $O(\log n)$ ；每条边被检查一次，

访问节点数 $/n \approx 2$ ；时间曲线 $T(n) \propto n$ ，为线性复杂度。

1.5 高阶函数和多态类型

高阶函数

高阶就是指其把其它函数当普通的“值”使用，可以作为参数传递，也能当成返回值。用于简化多参数的函数操作或者同类型批量数据的不同函数操作。比如，假设让 `trav` 把访问节点这个动作的函数当成参数，就能通过参数更换不同的遍历策略。抑或可以把比较函数作为参数来支持 `treecompare` 多态化，从而让 `heapify` 也能成为泛型，支持自定义排序，不需要再写一遍下沉逻辑。也就是像换比较规则、换输出格式，都只要改一个函数参数，主算法无需改变了。优点是代码复用，同时编译器的类型检查也会派上用场。

多态类型

所谓多态即多种形态，判断方法就是类型推导后剩下一些无约束的类型，则声明就是多态的。实例上就像 $\text{split} : \alpha \text{ list} \rightarrow \alpha \text{ list} \times \alpha \text{ list}$ ， α 可用任意类型替代，做到了完全多态，与元素类型无关，保证可复用于任何种类的列表。其优点在于可以避免写功能冗余的代码、便于维护。

二、 函数式拓展学习调研

2.1 函数式程序应用场景

编译器构造：因为代数数据类型 + 模式匹配让抽象语法树处理极其自然，比如 SML/NJ、OCaml 等主流编译器就是采用的自举方案。

金融定价引擎：基于函数式语言使用纯函数的原则，其无副作用，便于数学推理与并行回测，比如 Haskell 就被多家投行用于衍生品脚本。

分布式数据处理：市面上 Spark 的 RDD 转换本质上就是 `map/filter/fold` 高阶组合，依赖不可变数据集实现容错重算。

2.2 函数式特征延伸

语言	函数式特征	目的
Java	Stream API + Lambda	声明式集合处理，降低并发 bug
C++	std::transform/std::accumulate+lambda	算法泛化，减少手写循环
Python	map/filter/functools.reduce	链式数据变换，提高表达力

2.3 高阶与多态函数

$\text{map} : ('a \rightarrow 'b) \rightarrow 'a \text{ list} \rightarrow 'b \text{ list}$

同一函数适用于任何元素类型，目的是实现元素级转换。

$\text{foldl} : ('a * 'b \rightarrow 'b) \rightarrow 'b \rightarrow 'a \text{ list} \rightarrow 'b$

将任意二元操作累积到列表，目的是统一实现 sum、concat、and 等模式。

$\text{curry} / \text{uncurry} : ('a * 'b \rightarrow 'c) \rightarrow 'a \rightarrow 'b \rightarrow 'c$

利用函数是一等公民的原则，完成参数元组与多参函数互相转换，目的是提升组合性。

三、 函数式编程学习建议与心得

在函数式的课程中，我一开始速通了其语法，发现大概两节课就能完全掌握，其高阶和多态特性用起来十分方便，使一些批处理、或者是两个核心有相同点的任务得到简化。同时对于函数式的核心逻辑，也比较好理解，上上下下无非是函数间的嵌套、传参和返回，只需多注意模式匹配时宁多毋少。但我也发现函数式对编译的要求很严格，比如类型的对应。

随后具体的函数编写上，除了 heapify 中的逻辑处理需要仔细考量，其他函数在互相的包含调用中实现起来都比较简单。主要的问题在于 heapify 的函数一开始编写时没有确定其正确与否的检测手段，因而心态上有些消极。后续编写实验报告时，才开始考虑自己编写函数去显式展现树的结构以及验证其是否具有最小堆的性质。实现起来其实不难，因此这也是我对课程的建议之一。如若教师端能够在 heapify 实验中保存一个函数对学生端的 heapify 进行有效性检验，以输出 true 为标准答案，或者是另增一个小题让学生编写检验函数，会使实验更高效一些。

另外，我发现在实验过程中利用 split 函数的输出构造的树有一些结构特征，

会导致对一些特殊情况——比如前文提到的我在 `heapify` 实验中犯下的错误无法被检验出来，而不得不通过自己手动构造一棵树来显现。所以我也建议课程设置中对 `heapify` 的检验可以更周全一些。